

# ■ Kivo Workspace: Quick Presentation & Q&A Prep Guide

---

This guide is structured to give you a complete, high-level understanding of Kivo Workspace in under 15 minutes. It contains quick, punchy explanations of **what** we used, **why** we used it, and **how** to answer common technical questions during your demo/presentation.

---

## 1. Core Concept: What is Kivo Workspace?

---

- **Elevator Pitch:** Kivo is a local-first, privacy-respecting desktop knowledge assistant. It allows users to ingest files (PDFs), copy-paste text, or paste YouTube links, and run natural language queries against them.
  - **Key Value Proposition: 100% Privacy & Offline Execution.** User data, credentials, and document indexes never leave their local machine. There are no cloud database costs or subscription dependencies.
- 

## 2. The Tech Stack (What & Why)

---

| Layer           | Technology                            | Why We Used It                                                                                                                                                                                                          |
|-----------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend UI     | Flutter (Compiled natively for macOS) | Compiles to native, GPU-accelerated macOS code. Provides smooth, high-fidelity UI animations, support for native desktop features, and a premium "Notion-like" aesthetic.                                               |
| Backend Service | Python (FastAPI)                      | Python has the richest ecosystem for AI/ML libraries (FAISS, ONNX, tokenizers). FastAPI is used because it is lightweight, asynchronous, and provides fast local REST endpoints ( <code>http://localhost:8000</code> ). |

|                  |                                       |                                                                                                                                                                    |
|------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local Database   | SQLite                                | A serverless, zero-configuration SQL database. Used to store workspace configurations, source metadata, and processed chunk text coordinates locally on disk.      |
| Vector Search    | FAISS (Facebook AI Similarity Search) | The fastest CPU-optimized vector library. Stores and queries vector embeddings locally without needing a heavy docker container or external cloud vector database. |
| LLM Orchestrator | Ollama (Local Service)                | Handles local hosting of lightweight open-source models (like <code>qwen2.5:1.5b</code> or <code>llama3.2:1b</code> ) with a clean local API interface.            |

---

### 3. RAG Pipeline Architecture (Retrieval-Augmented Generation)

---

When a document (e.g., a PDF) is added, the following happens:

```
graph TD
  A[Source Document / Paste] --> B[Text Extraction]
  B --> C[Parent-Child Chunking]
  C --> D[Embeddings Generator: INT8 ONNX]
  D --> E[Store in FAISS Vector DB]
  E --> F[Query Time]
  F --> G[Search Child Chunks]
  G --> H[Reconstruct Parent Chunks]
  H --> I[LLM Context Generation]
```

#### ■ The Chunking Strategy: Parent-Child Chunking

- **What is it?** Instead of just slicing text into equal blocks, we split the document into:
- **Parent Chunks** (~1000 characters): Larger paragraphs containing the complete contextual detail.
- **Child Chunks** (~750 characters): Smaller, overlapping sentences nested within the parents.

- **Why did we do it?**
  - **The Problem:** Small chunks give precise search matches but lack context for the LLM. Large chunks contain context but dilute specific keywords during vector search.
  - **Our Solution:** We index and search on **Child Chunks** (ensuring high retrieval accuracy). However, when returning results, we feed the **Parent Chunks** (the full context) to the LLM. This provides the best of both worlds.
- 

## 4. The Embedding Model & 8-bit Quantization (High-Frequency Jury Questions)

---

### Q: What embedding model are you using?

- **Answer:** `Alibaba-NLP/gte-multilingual-base`.
- **Why?** It is a state-of-the-art multilingual model with a context length of 512 tokens. It ranks highly on the MTEB (Massive Text Embedding Benchmark) leaderboard and provides excellent semantic retrieval in English and other languages.

### Q: What is "8-bit INT8 Quantization" and why did you do it?

- **Answer:**
- **The Concept:** Standard AI models store weights as 32-bit floating-point decimals (FP32). **Quantization** is the process of converting these decimals to 8-bit integers (INT8).  
**Why?**
  1. **Speed (CPU Optimization):** It accelerates model calculations, making embedding generation **4x to 5x faster** on a standard CPU.
  2. **RAM & Disk Savings:** It shrinks the embedding model size from **~300MB down to ~75MB**. This allows Kivo to run lightweight in the background without consuming your system's memory.
  3. **No GPU Required:** Standard users don't have high-end AI graphics cards. INT8 ONNX runs extremely fast on basic consumer CPUs.

### Q: How did you implement Quantization?

- **Answer:** We converted the PyTorch `SentenceTransformer` model to **ONNX (Open Neural Network Exchange)** format and ran integer quantization using `onnxruntime.quantization`.

---

## 5. Comprehensive Q&A Bank for Mentors & Jurors

---

[!TIP] Keep your answers concise, practical, and focused on **User Experience** and **Local Performance**.

### Category A: General Architecture & Stack

#### ■ Q1: Why did you choose a local-first architecture instead of using OpenAI / Pinecone APIs?

- **Answer:** "Privacy, cost, and reliability. Corporate users and individuals do not want to upload sensitive documents to third-party cloud servers. A local-first architecture guarantees data privacy, costs exactly \$0 in API fees, and works 100% offline (e.g., during flights or areas with poor internet connection)."

#### ■ Q2: Why is there a separate Flutter frontend and a Python backend?

- **Answer:** "Flutter gives us a beautiful, compiled, and highly responsive user interface that integrates natively with macOS. Python is the industry standard for AI, vector algebra, and text parsing. By separating them, the Flutter app remains lightweight while talking to a high-performance Python FastAPI service running locally on port 8000."

#### ■ Q3: What happens if port 8000 is occupied on the user's system?

- **Answer:** "The backend port is configured dynamically via the environment settings. In production, the system checks for port availability and can bind to a random free port, which is then passed as an environment variable to the Flutter app during startup."

#### ■ Q4: How is data stored on disk? What is the database layout?

- **Answer:** "We store workspace data in a clean directory structure under the user's home folder (~/.kivo\_workspace or ~/Library/Application Support/KivoWorkspace). It contains:
- A central SQLite database (kivo.db) storing metadata for workspaces, active sources, and user configurations.
- A vector\_index/ folder containing FAISS index binary files for vector similarity lookup.
- A sources/ folder which stores the raw imported documents."

---

### Category B: RAG, Indexing & Vector Search

■ Q5: Why use FAISS instead of other vector databases like ChromaDB, Milvus, or Qdrant?

- **Answer:** "ChromaDB, Qdrant, and Milvus are excellent, but they are either cloud-based or run as standalone service containers (like Docker), which is too heavy for a simple desktop app. FAISS is a lightweight C++ library that compiles directly into Python. It writes index files directly to disk (like a SQLite file), making it perfect for embedded local applications."

■ Q6: How do you handle search index persistence? If I close the app, does the vector index vanish?

- **Answer:** "No. FAISS provides native save and load methods (`faiss.write_index` and `faiss.read_index`). Whenever a new document is ingested, the index is computed in RAM and written as an `.index` binary file in the workspace directory. On app startup, the backend automatically reads the file back into memory."

■ Q7: Why did you implement Parent-Child chunking instead of simple chunking?

- **Answer:** "Simple chunking either misses context (if chunks are too small) or dilutes specific facts (if chunks are too large). Parent-Child chunking searches on small, precise child sentences to find the exact match, but feeds the larger parent paragraph to the LLM so it has the surrounding context to generate a correct answer."

■ Q8: What embedding dimensions does the model produce, and how do you compare vectors?

- **Answer:** "Our GTE model generates **768-dimensional vectors**. We normalize these vectors and compute similarity using the Inner Product (IP) metric in FAISS, which is mathematically equivalent to **Cosine Similarity** (since the vectors are L2-normalized). This measures the directional alignment of semantic concepts."

■ Q9: How do you handle context length constraints of local LLMs?

- **Answer:** "We implemented **Context Budgeting**. We dynamically reconstruct parent chunks and cap the context length at 2000 tokens. This ensures the local LLM (qwen2.5:1.5b) responds within seconds and doesn't run out of memory or experience latency degradation."

---

## Category C: Ingestion & Document Processing

■ Q10: How do you extract text from PDF files locally?

- **Answer:** "We use Python libraries (such as `pdfplumber` or `pypdf`) to parse the PDF structure locally. We clean formatting anomalies, strip extra spaces, and pass the normalized raw text stream directly to our Parent-Child text processor."

#### ■ Q11: What happens when a user pastes a YouTube URL? How does it work offline?

- **Answer:** "The YouTube ingestion requires an active internet connection *only* during importing. The backend extracts the video ID and uses a lightweight API to fetch the video's official transcriptions. Once downloaded, the transcription text is processed, embedded, and stored locally. Subsequent querying of that video works 100% offline."

#### ■ Q12: How do you handle YouTube videos that do not have captions/subtitles?

- **Answer:** "If official transcripts are not available, the backend falls back to using local Whisper models (via `faster-whisper` on CPU) to transcribe the audio into text, though for standard demos, we recommend pasting videos with pre-existing subtitles to save processing time."

#### ■ Q13: What happens during "Copy Text" ingestion?

- **Answer:** "The user pastes raw text into the Flutter UI. Flutter sends a POST request with the text body to our FastAPI backend. The backend processes the string just like a text file, splits it into parent-child chunks, computes embeddings via ONNX, and appends them to the active workspace's FAISS index."

---

### Category D: Performance & LLMs

#### ■ Q14: Why choose Qwen 2.5 (1.5B) or Llama 3.2 (1B) over larger models like Llama 3 8B?

- **Answer:** "A 7B or 8B model requires 8GB to 16GB of dedicated VRAM to run smoothly, which would freeze standard laptops. A 1.5B model runs comfortably on standard CPU RAM (requiring less than 2GB), responds in under 3 seconds locally, and is highly optimized to generate rich Markdown text (tables, code blocks, bold text)."

#### ■ Q15: How does the application perform during parallel requests or heavy ingestion?

- **Answer:** "The backend is built using FastAPI (asyncio) and runs uvicorn workers. During heavy ingestion (embedding generation), calculations are run using a thread-pool executor to prevent blocking the main event loop, allowing the UI to show a smooth progress bar without freezing."
-

## Category E: Advanced Scenarios & Troubleshooting

### ■ Q16: How do you update or delete a document's vector embeddings?

- **Answer:** "When a user deletes a source, we delete the source row and its associated chunks from the SQLite database. Since FAISS does not natively support easy index deletion by ID, we simply filter out the deleted chunks in SQLite and rebuild the FAISS index from the remaining vector arrays saved on disk, then overwrite the `.index` file."

### ■ Q17: What is ONNX?

- **Answer:** "ONNX stands for **Open Neural Network Exchange**. It is an open-source framework designed to represent machine learning models. By converting our PyTorch model to ONNX, we decouple it from heavy Python libraries and execute it using `onnxruntime`, which is written in C++ and highly optimized for CPU threading."

### ■ Q18: What happens if the user imports two identical documents?

- **Answer:** "FAISS indexes both documents independently because each chunk gets a unique UUID mapping in the SQLite database. During similarity search, if both match, they will appear in the top-K list. Our system automatically de-duplicates overlapping parents to avoid sending redundant text to the LLM."

### ■ Q19: Why use Ollama instead of raw Hugging Face transformers pipeline in Python?

- **Answer:** "Ollama is written in Go and uses llama.cpp under the hood, which uses advanced hardware-level optimizations (like Metal on macOS or AVX2 on Windows). It runs as a low-overhead service and manages active model memory (unloading models when idle), whereas running models natively in Python via PyTorch/Transformers keeps the memory blocked and consumes significantly more RAM."

### ■ Q20: What is the limitation of this local RAG setup, and how can it be expanded?

- **Answer:** "The main limitations are **scanned document OCR** (which requires heavier local computing) and **extremely large vector counts** (where flat index search could slow down). It can be expanded by integrating local OCR libraries (like Tesseract) and using Hierarchical Navigable Small World (HNSW) indexing in FAISS for sub-millisecond search on millions of vectors."

---

## 6. How to Demo the "Copy Text" Feature Successfully

---

1. **Reset Workspace:** Clean up any past logs or onboarded state if you want a fresh setup.
2. **Launch Backend & App:** Ensure your backend terminal is running uvicorn on port `8000`, and Flutter is running.
3. **Pasting the Text:**
4. Click **Add Source** -> Select **Copy Text**.
5. Paste the 20-paragraph AI History text (provided in your chat history).
6. Name the source (e.g., "History of AI").
7. **Showing off the Querying:**
8. Ask an exact question: *"Who proposed the Turing Test?"* (Expected answer: Alan Turing in 1950).
9. Ask a complex multi-hop question: *"Compare symbolic AI and machine learning approaches."*  
(This demonstrates the Parent-Child retrieval matching both the rule-based 1960s-1970s era and the 1990s ML era).
10. **Highlight Formatting:** Show that the response has headers, bullet points, or bold text, making it highly readable.